

# Technical Specification Report

## AI-Based Face Recognition System using InsightFace (Buffalo\_L)

|             |                                                        |
|-------------|--------------------------------------------------------|
| Project     | Smart Attendance Management System                     |
| Module      | Face Detection & Face Recognition                      |
| Framework   | InsightFace                                            |
| Model       | Buffalo_L                                              |
| Language    | Python                                                 |
| Integration | FastAPI + Qdrant Vector Database + OpenCV + PostgreSQL |

### 1. Objective

The objective of this module is to automatically detect and recognize students from uploaded classroom images for attendance marking.

The system is designed to:

- Detect multiple faces in a single image.
- Generate facial embeddings for each detected face.
- Compare embeddings against registered student embeddings.
- Identify recognized students using vector similarity search.
- Automatically update attendance records.
- Save annotated images with recognized faces.
- Calculate attendance accuracy.

### 2. Technology Stack

| Component            | Technology  |
|----------------------|-------------|
| Face Detection       | InsightFace |
| Face Recognition     | Buffalo_L   |
| Embedding Generation | ArcFace     |
| Image Processing     | OpenCV      |
| Backend API          | FastAPI     |
| Database             | PostgreSQL  |
| Vector Database      | Qdrant      |

| Component            | Technology |
|----------------------|------------|
| Programming Language | Python     |

### 3. InsightFace

InsightFace is an open-source deep learning framework for face analysis that provides high-performance models for face detection, face alignment, and face recognition. The framework is optimized for both CPU and GPU execution, making it suitable for real-time face recognition applications.

The model is initialized using the following code:

```
from insightface.app import FaceAnalysis

app = FaceAnalysis(name="buffalo_l")
app.prepare(ctx_id=0)
```

### 4. Buffalo\_L Model

The system uses the Buffalo\_L pretrained model package available in InsightFace.

Buffalo\_L contains multiple ONNX models that work together to perform face recognition. The package includes:

- Face Detection Model
- Facial Landmark Detection Model
- Face Recognition (ArcFace) Model

These models are automatically loaded when the FaceAnalysis object is initialized.

### 5. Model Initialization

The following code initializes the model:

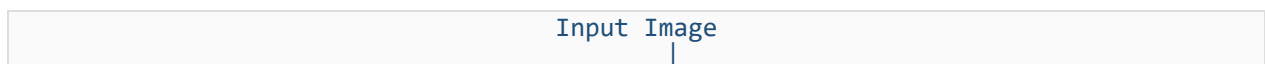
```
from insightface.app import FaceAnalysis

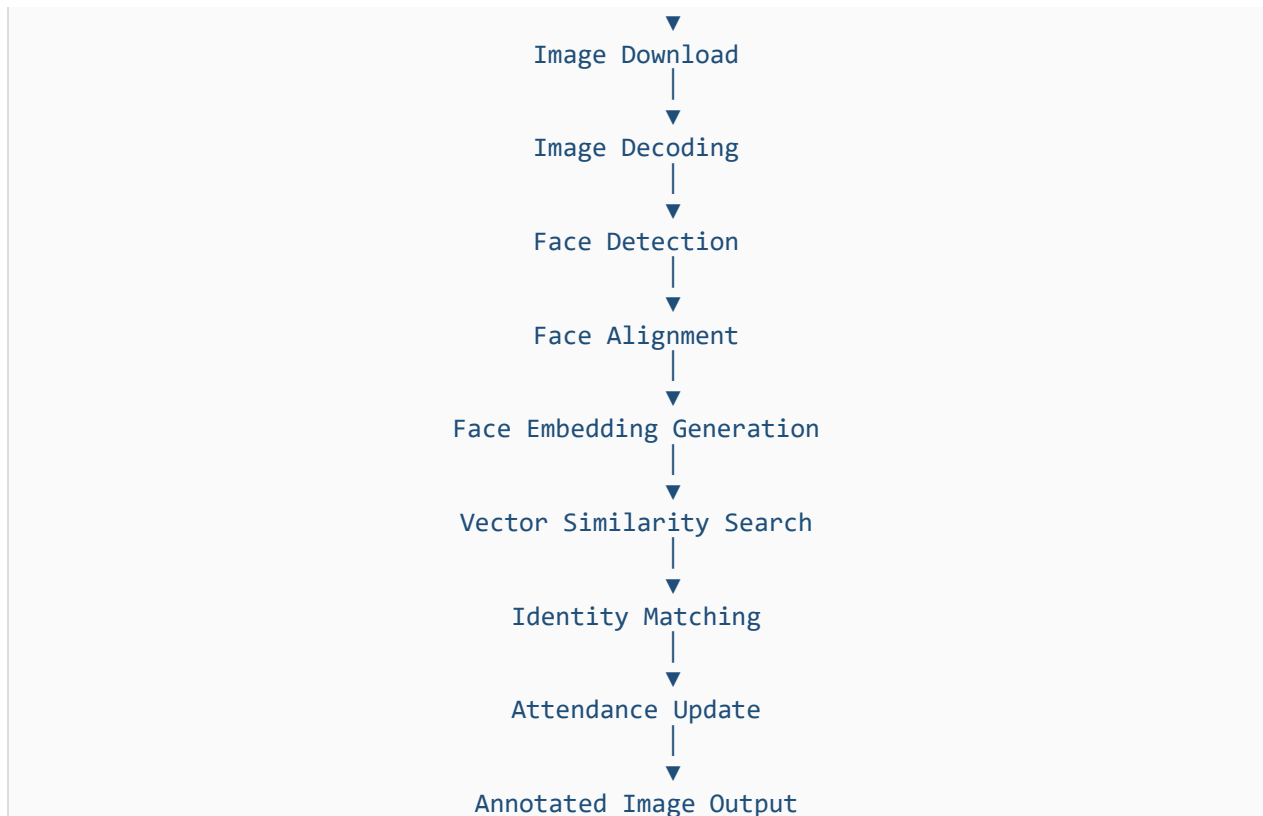
app = FaceAnalysis(name="buffalo_l")
app.prepare(ctx_id=0)
```

Explanation:

- name="buffalo\_l" loads the Buffalo\_L model package.
- ctx\_id=0 enables GPU execution using CUDA.
- ctx\_id=-1 enables CPU execution if GPU is unavailable.

### 6. Face Detection Pipeline





## 7. Face Detection

The uploaded classroom image is downloaded and decoded using OpenCV.

```
img = cv2.imdecode(...)
```

Faces are detected using:

```
faces = app.get(img)
```

For every detected face, the model returns:

- Bounding Box
- Facial Landmarks
- Face Embedding
- Detection Confidence

## 8. Face Alignment

Before recognition, each detected face is automatically aligned using facial landmarks.

Face alignment improves recognition accuracy by compensating for:

- Head rotation
- Camera angle

- Face tilt
- Scale variation

## 9. Face Embedding Generation

Each aligned face is converted into a 512-dimensional embedding vector.

```
embedding = face.embedding
```

Characteristics of the embedding:

- Fixed-length vector (512 dimensions)
- Represents unique facial features
- Independent of image resolution
- Used for vector similarity search

The generated embeddings are stored in Qdrant.

## 10. Student Registration

During student registration:

1. Multiple student images are collected.
2. Face embeddings are generated from each image.
3. All embeddings are averaged.
4. The average embedding is stored in Qdrant.

Using multiple images improves recognition accuracy under different:

- Lighting conditions
- Facial expressions
- Viewing angles

## 11. Vector Database (Qdrant)

Qdrant stores the facial embeddings of registered students.

Each record contains:

- Point ID
- Student ID
- Student Name
- 512-dimensional face embedding

Qdrant performs high-speed nearest-neighbor search to identify the closest matching face during recognition.

## 12. Face Recognition

For every detected face:

```
matches = search_face(...)
```

The generated embedding is compared against all registered embeddings stored in Qdrant.

The search returns:

- Closest matching student
- Similarity score

The student with the highest similarity score is selected as the recognized identity.

### 13. Similarity Thresholds

The system uses confidence thresholds to determine recognition results.

| Similarity Score | Result                | Bounding Box Color |
|------------------|-----------------------|--------------------|
| $\geq 0.70$      | High Confidence Match | Green              |
| $\geq 0.50$      | Possible Match        | Yellow             |
| $< 0.50$         | Unknown Person        | Red                |

### 14. Attendance Processing

If the similarity score exceeds the recognition threshold:

- Student status is updated from ABSENT to PRESENT
- Confidence score is stored
- Attendance timestamp is recorded
- Annotated image URL is saved

Attendance records are automatically updated in PostgreSQL.

### 15. Image Annotation

Detected faces are annotated using OpenCV.

The following information is displayed:

- Student Name
- Similarity Score
- Bounding Box

Example:

```
John Smith (0.86)
```

The processed image is saved for future verification.

## 16. Performance Metrics

The implementation records execution time for:

- Image Download
- Image Decoding
- Face Detection
- Vector Similarity Search
- Image Saving
- Database Operations
- Total Processing Time

These metrics help evaluate and optimize system performance.

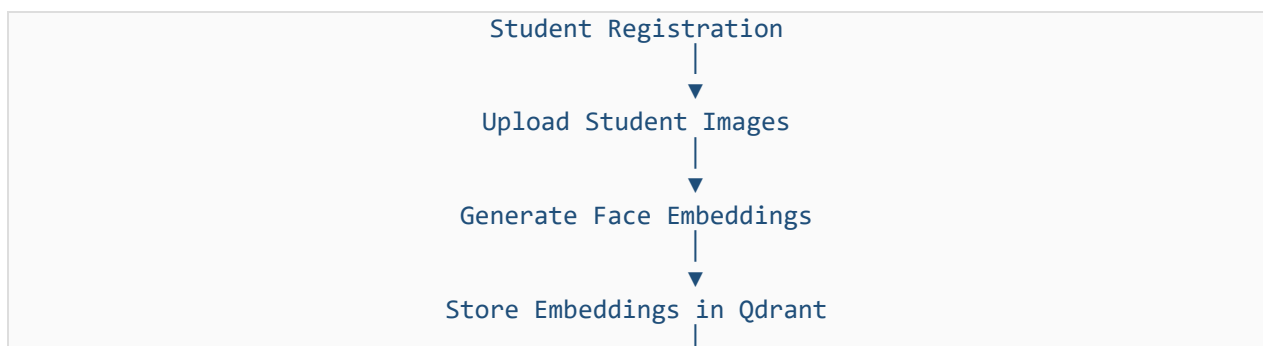
## 17. Advantages

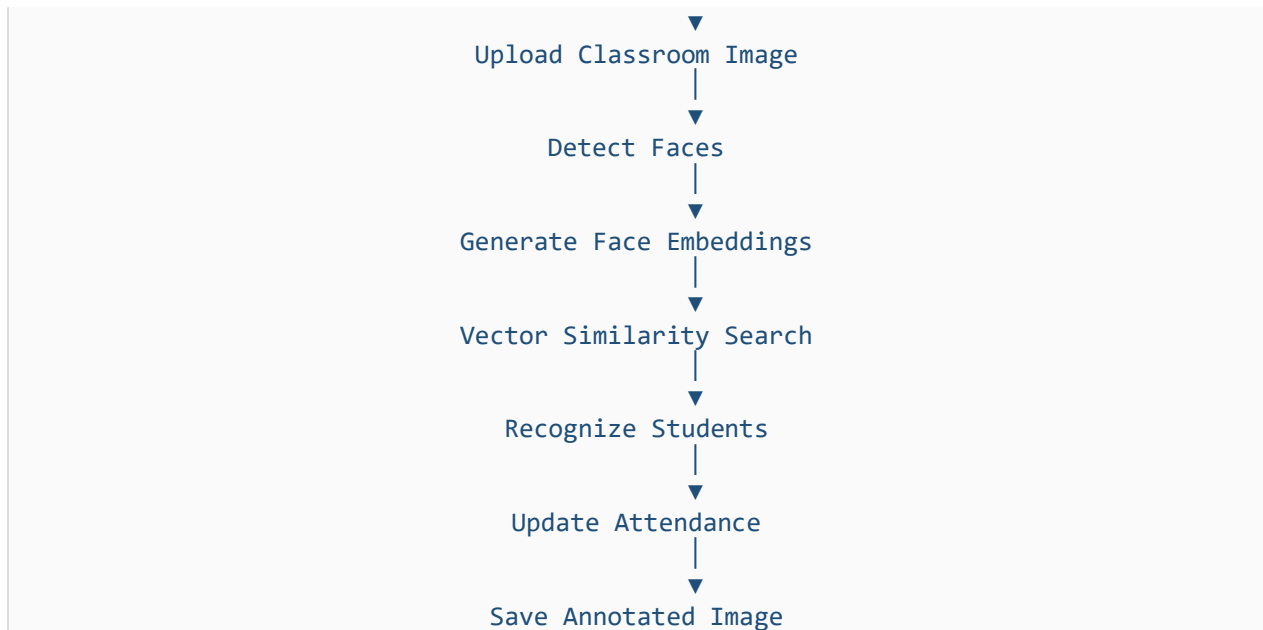
- Real-time face recognition
- Supports multiple face detection
- High recognition accuracy
- GPU acceleration support
- Fast vector similarity search using Qdrant
- Scalable architecture
- Automatic attendance generation
- Easy integration with FastAPI

## 18. Limitations

- Recognition quality depends on image resolution.
- Heavy face occlusion may reduce recognition accuracy.
- Poor lighting conditions can affect face detection.
- CPU execution is slower than GPU execution.
- Similar-looking individuals may require threshold tuning.

## 19. System Workflow





## 20. Conclusion

The Smart Attendance Management System uses the InsightFace Buffalo\_L model to perform accurate face detection and recognition. The model generates 512-dimensional facial embeddings for each detected face, which are compared with registered student embeddings stored in Qdrant using vector similarity search. Recognized students are automatically marked present, attendance records are updated in PostgreSQL, and annotated images are generated for verification. The architecture is scalable, supports real-time processing, and provides an efficient AI-based solution for automated classroom attendance.